Linux RHC030

Administración de Procesos

Administración de Procesos

- Introducción a Procesos
- Estados del proceso
- Programación de procesos: nice y renice
- Envío de señales
- Control de trabajo
- Programación de tareas retrasadas: at
- Programación de tareas periódicas: cron

© 2007 Linux. Todos los derechos reservados.

Introducción a Procesos

- **Conceptos Claves**
  - Un proceso es una instancia de un ejecutable en ejecución identificado por un id de proceso (pid).
  - Debido a que Linux implementa memoria virtual cada proceso posee su propio contexto distintivo de memoria.
  - Un proceso tiene un uid y una colección de gid como credenciales.
  - Un proceso tiene un contexto de sistema de archivos incluyendo un cwd, una umask, un directorio raíz y una colección de archivos abiertos.
  - Un proceso tiene un contexto de programación que incluye un valor de niceness.
  - Un proceso tiene una colección de variables de entorno.
  - El comando **ps** puede utilizarse para examinar todos los procesos actuales en ejecución.
  - El comando **top** puede utilizarse para controlar todos los procesos en ejecución.

© 2007 Linux. Todos los derechos reservados.

Procesos

- Un proceso es una instancia de un programa en ejecución
- Clases de programas que:
  - Diseñados para iniciar rapidamente,
  - Especializados en función,
  - Trabajan bien con otros programas
  - Usualmente no tienen una contraparte en otros sistemas operativos
- La responsabilidad del kernel de Linux es proporcionar un lugar para que los procesos ejerzan su labor sin necesidad de tocar el campo del otro.

© 2007 Linux. Todos los derechos reservados.

¿ Que es un Proceso?

- **Contexto de ejecución**
  - Cada proceso existe (al menos hasta cierto punto) dentro de una memoria física de la máquina. Puesto que Linux (y Unix) están diseñados para ser un entorno multiusuario, la memoria asignada a un proceso está protegida y ningún otro proceso puede acceder a ésta.
- **Contexto de E/S**
  - Cada proceso interactúa hasta cierto punto con el sistema de archivos para leer y escribir información que existe o existirá después del ciclo de vida del proceso.
- **Variables de entorno**
  - Cada proceso mantiene su propia lista de pares nombre-valor, conocida como variables de entorno o en general como el *entorno* del proceso.

© 2007 Linux. Todos los derechos reservados.

¿ Que es un Proceso?

- **Información de Herencia**
  - Cada proceso se identifica con un PID o id del proceso asignado en el momento de su creación.
- **Credenciales**
  - Cada proceso se ejecuta bajo el contexto de un determinado usuario (o más exactamente de un id de usuario determinado) y bajo el contexto de una colección de id de grupo (generalmente, todos los grupos a los que pertenezca el usuario).
- **Estadísticas de recursos y límites**
  - Cada proceso también registra estadísticas para trazar la cantidad de recursos del sistema utilizados, el número de archivos abiertos, la cantidad de tiempo de CPU, etc.

© 2007 Linux. Todos los derechos reservados.

Utilitarios de Administración de Procesos

- Los sistemas GNU/Linux cuentan varios programas para efectuar el seguimiento de los procesos que se están ejecutando en el sistema. Entre los mas usados en la interfase de texto están los programas **ps** y **top**.
- **Ps:** Sin ninguna opción dará la lista de procesos que están corriendo desde la terminal donde se ejecuto el **ps. .**

```
[shrek@pantano:~]$ ps
  PID TTY          TIME CMD
 9648 tty2     00:00:02 bash
 9659 tty2     00:00:00 ps
[shrek@pantano:~]$
```

PID: Id del Proceso

TTY: identifica la consola donde se esta ejecutando el proceso.

TIME: tiempo total del proceso que se ha estado ejecutando.

© 2007 Linux. Todos los derechos reservados.

Utilitario de Administración de Procesos (PS)

- Si añadimos la opción l tendremos un listado largo del comando **ps**. En algunas versiones se usa la opción -l.

```
[shrek@pantano:~]$ ps l
F   UID   PID  PPID PRI  NI   VSZ  RSS  WCHAN STAT TTY   TIME COMMAND
4   100  9648     1   9   0  4368 1400 llbldO S    tty2  0:01 -bash
4   100  9660  9648  17   0  2676  732      - R    tty2  0:00 ps l
```

UID : que identifica el dueño del proceso
PID : ID del Proceso
PPID : PID del proceso padre
NI : nivel que se otorga a un proceso para requerir cierto privilegio
VSZ :es el tamaño que tiene el proceso.
RSS :es la tamaño del proceso que se encuentra residente en la memoria.
WCHAN:es el nombre de la función del kernel donde el proceso esta durmiendo.

© 2007 Linux. Todos los derechos reservados.

Visualización de Procesos

- El comando **ps** lista todos los procesos iniciados desde una terminal de usuario.

Opciones comunes del comando ps para la selección de salida

Opciones comunes del comando ps para la selección de procesos

Opción	Formato de la salida
-f	listado "completo"
-l, l	formato largo
-j, j	formato de trabajos
-o, o, --formato *str*	formato definido del usuario utilizando campos especificados por *str* (campos disponibles para *str* se pueden listar con **ps L** o consultando la página de manual **ps (1)**).
-H	Muestra la jerarquía del proceso
f, --forest	Muestra la jerarquía de proceso incluyendo las representaciones ASCII
h	No imprime las líneas del encabezado
-w	salida "ancha" (incluye nombres de los comandos más largos)

© 2007 Linux. Todos los derechos reservados.

Utilitario de Administración de Procesos (TOP)

- El comando **ps** nos muestra una radiografía de los procesos en el momento, pero no nos muestra los cambios que se van teniendo.
- Para esto contamos con el comando **top**. El mismo muestra en tiempo real la situación de los procesos que se están ejecutando en el sistema, ordenados por defecto según el porcentaje la CPU que estén usando.
- Al ejecutarlo se podrá ver otra información adicional, como la cantidad de usuarios que están en el sistema, cuantos procesos están corriendo y de estos cuantos estas activos, cuantos durmiendo, cuantos en proceso de terminar (ZOMBIE) y cuantos finalizados.

© 2007 Linux. Todos los derechos reservados.

Utilitario de Administración de Procesos (TOP)

- Además se podrá ver la cantidad e memoria física total, la cantidad usada y la cantidad libre; así como también se podrá obtener la misma información de la memoria swap.
- Lo más importante es que esta información se ira actualizando automáticamente cada tanto tiempo, por defecto 5 segundos, y que podremos ir alterando lo que va mostrando.

© 2007 Linux. Todos los derechos reservados.

Monitoreo de Procesos

- El comando **top** mostrará un cuadro de procesos actualmente en ejecución, que se actualiza ciertos segundos

```
17:46:38  up  4:28,  7 users,  load average: 0.07, 0.02, 0.00
101 processes: 100 sleeping, 1 running, 0 zombie, 0 stopped
CPU states:   3.4% user   1.0% system   0.0% nice   0.0% iowait  95.4% idle
Mem:   255232k av,  232796k used,   22436k free,       0k shrd,   32404k buff
                    146208k actv,   33884k in_d,    4716k in_c
Swap:  522104k av,   88368k used,  433736k free                  104280k cached

  PID USER     PRI  NI  SIZE  RSS SHARE STAT %CPU %MEM   TIME CPU COMMAND
 1150 einstein  15   0  4468 4108  2252 S     2.9  1.6   0:51   0 metacity
 1186 einstein  15   0  3132 2112  1436 S     0.9  0.8   0:04   0 battstat-appl
 3057 einstein  15   0 19596  18M 12356 S     0.9  7.5   0:06   0 galeon-bin
 3622 maxwell   22   0  1088 1088   856 R     0.9  0.4   0:00   0 top
    1 root      15   0   108   76    52 S     0.0  0.0   0:04   0 init
    2 root      15   0     0    0     0 SW    0.0  0.0   0:00   0 keventd
    3 root      15   0     0    0     0 SW    0.0  0.0   0:00   0 kapmd
    4 root      34  19     0    0     0 SWN   0.0  0.0   0:00   0 ksoftirqd_CPU
```

© 2007 Linux. Todos los derechos reservados.

El comando top

Comandos top más utilizados

Presionar tecla	Comando
P	Clasificar procesos por actividad de la CPU (procesador)
u	Reducir procesos visualizados pertenecientes a un usuario específico
k	Matar un proceso (enviar una señal a un proceso)
r	renice un proceso

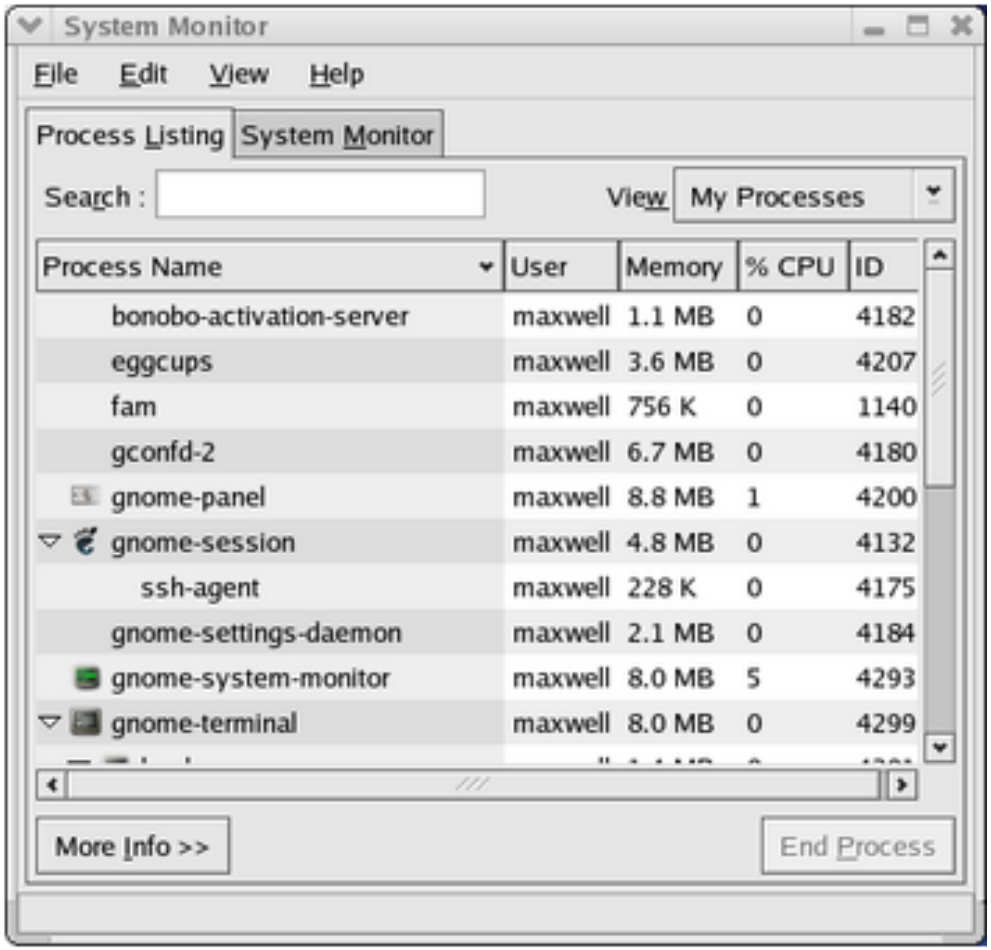
Opciones para el comando top

Opción	Efecto
-d *segundos*	Demora *segundos* segundos entre actualizaciones (por defecto = 5 segundos).
-q	Actualiza tan pronto como sea posible.
-n *N*	Se ejecuta para iteraciones *N*, luego sale.
-b	Se ejecuta en "modo por lote" simplemente como si escribiera a una terminal tonta.

© 2007 Linux. Todos los derechos reservados.

Monitor de Sistema GNOME

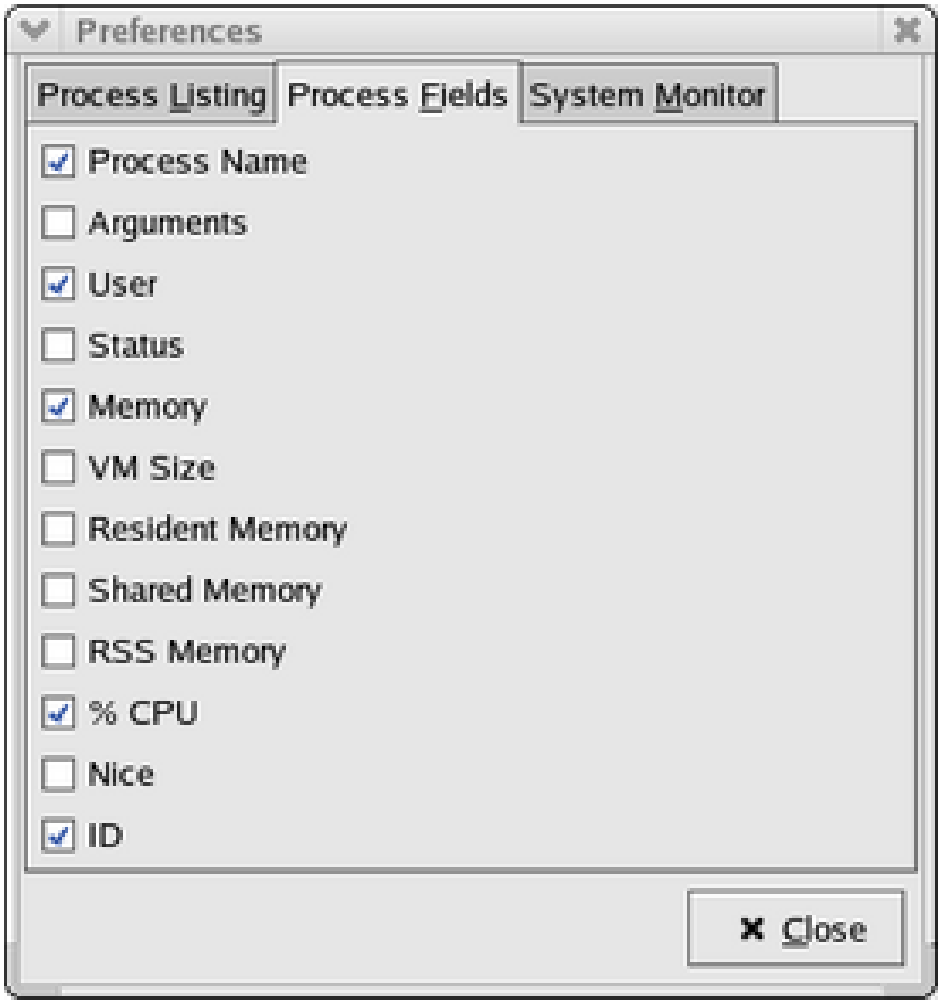
- La aplicación se puede iniciar desde una línea de comandos como:
  - **gnome-system-monitor**
  - Menú Herramientas: monitor del sistema.



© 2007 Linux. Todos los derechos reservados.

Configurando mostrar los campos en el Monitor de Sistema GNOME

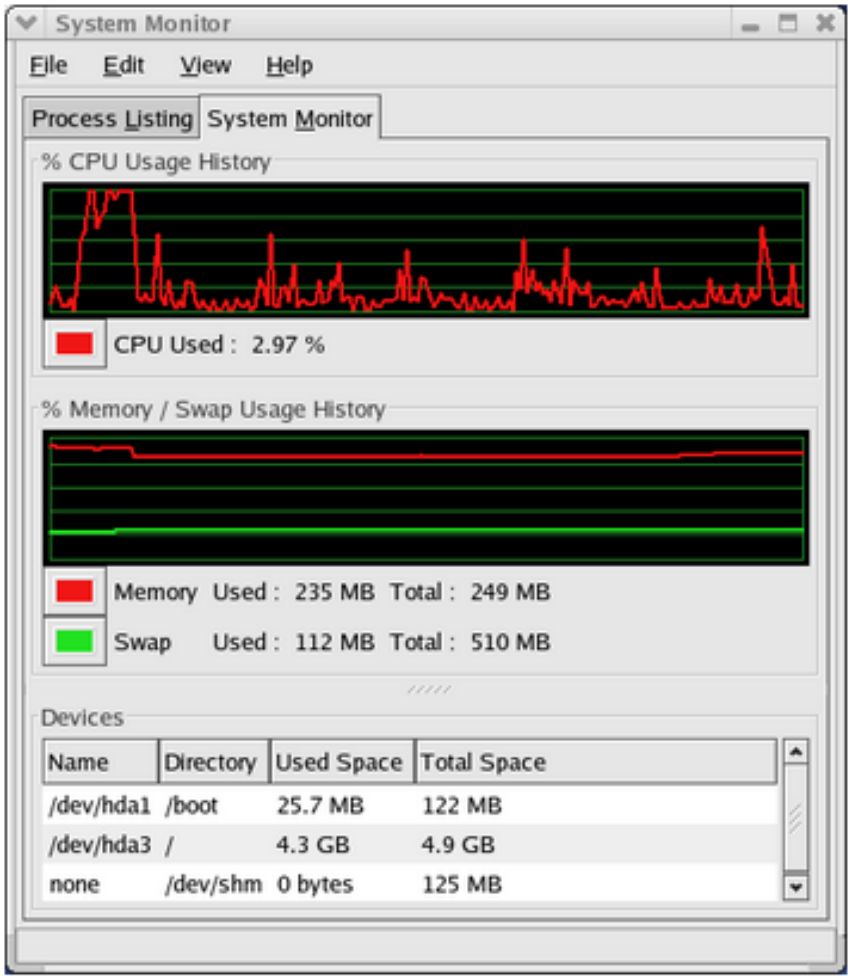
- Monitor de Sistema puede ser configurado abriendo:
  - Editor:Preferencias



© 2007 Linux. Todos los derechos reservados.

Panel del Monitor de Sistema

- El monitor de Sistema provee un segundo panel, que muestra:
  - Grafica de Utilización total de CPU
  - Utilización de Memoria versus tiempo
  - Tabla de uso de disco



© 2007 Linux. Todos los derechos reservados.

Localizando Procesos

- **pgrep** permite a los usuarios listar rápidamente procesos por:
  - Nombre de Comando,
  - Usuario,
  - Terminal, o
  - Grupo

Opciones comunes para especificar el formato de salida pgrep

Opciones comunes para especificar el criterio de selección del proceso pgrep.

Opción	Efecto
-d *delimitador*	Usa *delimitador* para delimitar cada ID de proceso (por defecto se utiliza una nueva línea).
-l	Lista el nombre del proceso y el ID del proceso.

© 2007 Linux. Todos los derechos reservados.

Estados del Proceso

- **Conceptos Claves**
  - En Linux, el primer proceso, **/sbin/init**, lo inicia el kernel en el arranque. Todos los demás procesos son el resultado de un proceso padre que se duplica o *bifurca*.
  - Un proceso comienza a ejecutar un nuevo comando a través de un proceso llamado *execing*.
  - Los nuevos comandos suelen ejecutarse mediante un proceso (a menudo una shell) primero mediante una bifurcación y luego ejecutando un nuevo comando. Este mecanismo se conoce como el mecanismo *fork y exec*.
  - Los procesos siempre pueden encontrarse en uno de los cinco estados: *ejecutable*, *dormido voluntario*, *dormido involuntario*, *detenido* o *zombi*.
  - La ascendencia del proceso puede verse con el comando **pstree**.
  - Cuando un proceso muere, el padre del proceso debe recolectar su información del código de retorno y del uso de recursos.
  - Cuando un padre muere antes que sus hijos, el primer proceso hereda los huérfanos (usualmente **/sbin/init**).

© 2007 Linux. Todos los derechos reservados.

Como se inician los procesos

- Nuevos procesos son creados a traves de un *forking*.
- Cuando un proceso se bifurca, este crea un duplicado de si mismo.
  - Inmediatamente después de una bifurcación, el proceso recién creado (el hijo) es un duplicado exacto del proceso original (el padre). El hijo hereda una copia idéntica de la memoria del proceso original, los archivos abiertos de padre, copias idénticas de los parámetros del padre, tales como el directorio de trabajo actual o umask.
  - El hijo tiene un ID de proceso diferente y un proceso de padre diferente

© 2007 Linux. Todos los derechos reservados.

Ejecución de un nuevo comando (Exec-ing)

- Los nuevos comandos se ejecutan a través de un técnica llamada *execing*
- Cuando se ejecuta un nuevo comando,
  - El proceso actual borra y libera la mayoría de sus recursos, y
  - Carga una nueva colección de instrucciones desde el comando especificado en el sistema de archivos.
- Algunos programas pueden bifurcarse (fork) sin ejecutar un nuevo comando (execing).
  - Demonios de Red,
    - Bifurcan un nuevo hijo para manejar una conexión de un cliente específico
    - El padre retorna para escuchar nuevos clientes

© 2007 Linux. Todos los derechos reservados.

El comando pstree

- Cada proceso a excepción del primero se crea por bifurcación, dentro de los procesos existe un linaje bien definido de relaciones padre e hijo.

```
[maxwell@station maxwell]$ pstree
init-+-apmd
     |-atd
     |-automount
     |-battstat-applet
       ...
     |-fetchmail
     |-galeon-bin
     |-gconfd-1
     |-2*[gconfd-2]
     |-gdm-binary-+-gdm-binary-+-X
     |            |            `-gnome-session---ssh-agent
     |            `-gdm-binary---gnome-session---ssh-agent
```

© 2007 Linux. Todos los derechos reservados.

Como muere un proceso

- Un proceso muere ya sea que muera normalmente seleccionando *exit* o anormalmente como el resultado de recibir una señal.
- **Cuando un procesos sale,**
  - Todos sus recursos se liberan, a excepción del código de retorno
  - Es responsabilidad del padre del proceso coleccionar esta información y liberar los últimos recursos del hijo muerto
- **Si el proceso padre muere antes que el proceso hijo nace**
  - **Huerfanos -** El hijo queda *huérfano*
  - **Zombies –** Se genera el estado Zombie, entre el momento en que un proceso sale, liberando la mayoría de sus recursos, y el momento en que su padre recoge su valor de retorno, liberando el resto de sus recursos,
- **Huerfanos son adoptados por el primer proceso,**
  - /sbin/init es un padre muy inteligente, ya que se encarga de adoptar hijos huerfanos

© 2007 Linux. Todos los derechos reservados.

Los 5 estados del Proceso

- El siguiente listado muestra los cinco estados, junto con la letra convencional utilizada por **ps**, **top** y otros comandos para identificar el estado actual de un proceso.

Estado	Descripción
Ejecutable (R)	Los procesos en un estado ejecutable son procesos que si tienen la oportunidad de acceder la CPU
Dormido voluntario (interrumpible) (S)	Por lo general, este es un proceso que no tiene nada que hacer hasta que suceda algo interesante. Un ejemplo clásico es el demonio de red **httpd**, el cual es un proceso que implementa un servidor de red. En medio de solicitudes de un cliente (navegador de red), el servidor no tiene nada que hacer, y elige irse a dormir.
Dormido involuntario (no interrumpible) (D)	Cuando dos procesos tratan de acceder el mismo recurso de sistema al mismo tiempo. En estas situaciones, el kernel fuerza al proceso a dormir.
Procesos detenidos (suspendidos) (T)	Ocasionalmente, los usuarios deciden suspender procesos. Los procesos suspendidos no realizarán ninguna acción hasta no ser reiniciados por el usuario. En la shell **bash**, la secuencia de teclas **CONTROL**-**Z** puede utilizarse para suspender un proceso.
Procesos zombi (Z)	cada proceso muriendo pasa a través de un estado zombi transitorio. No obstante, en ocasiones, algunos se quedan en ese estado.

© 2007 Linux. Todos los derechos reservados.

Programación de Procesos: nice y renice

- **Conceptos Claves**

  - Una tarea primaria del kernel de Linux es la programación de procesos.
  - Cada proceso tiene un valor de niceness que influye en su programación.
  - Los comandos **nice** y **renice** pueden cambiar la prioridad de programación de un proceso.

© 2007 Linux. Todos los derechos reservados.

Programación de Procesos

- Cada proceso tiene dos valores que influyen en su programación.
  - (dinamica) valor de *prioridad* del proceso
  - (fija) valor del *niceness* del proceso

```
00:25:43  up 11:07, 10 users,  load average: 6.02, 5.08, 3.01
128 processes: 121 sleeping, 7 running, 0 zombie, 0 stopped
CPU states:   8.5% user   3.2% system   4.6% nice   0.0% iowait  83.5% idle
Mem:   255232k av,  251560k used,    3672k free,       0k shrd,   36592k buff
                    162724k actv,   33644k in_d,    3952k in_c
Swap:  522104k av,  145148k used,  376956k free                   74780k cached

  PID USER     PRI  NI  SIZE  RSS SHARE STAT %CPU %MEM   TIME CPU COMMAND
 6698 maxwell   25   0   404  400   352 R    19.3  0.1   0:24   0 cat
 6699 maxwell   22   0   400  400   352 R    19.3  0.1   0:24   0 cat
 6700 maxwell   24   0   400  372   352 R    19.3  0.1   0:24   0 cat
 6757 einstein  21   0 25992  24M  3596 R    17.4  9.7   0:04   0 jade
 6701 maxwell   25   0   400  400   352 R    16.4  0.1   0:24   0 cat
 6686 maxwell   35  10  6608 4836  1108 R N   4.8  1.8   0:44   0 xlyap
 6758 maxwell   19   0  1120 1120   852 R     2.9  0.4   0:00   0 top
```

© 2007 Linux. Todos los derechos reservados.

Prioridad de Procesos

- El kernel da a cada proceso una cantidad de contadores.
- Cada vez que un proceso se programa en la CPU, entrega uno de sus contadores.
- Cuando decide qué proceso programar en la próxima CPU, el kernel escoge un proceso ejecutable con la mayoría de contadores.

© 2007 Linux. Todos los derechos reservados.

Proceso Niceness

- Cada proceso también tiene un valor estático conocido como su valor de niceness.
- El valor tiene un rango que va de -20 a 19 para cualquier proceso, iniciando en 0 por defecto.
- Aquellos con un mayor valor de niceness (>0)
  - obtienen menos contadores
  - menos tiempo en la CPU
- Aquellos con un valor niceness (< 0)
  - Obtienen mas contadores
  - mas tiempo de CPU

© 2007 Linux. Todos los derechos reservados.

Iniciar un comando con prioridad baja

- El comando **nice** se utiliza para establecer un valor de niceness del proceso al iniciar el proceso.

```
[maxwell@station maxwell]$ nice -19 ./simulation &
```

```
PID USER     PRI  NI  SIZE  RSS SHARE STAT %CPU %MEM   TIME CPU COMMAND
7192 maxwell   25   0   400  400   352 R    22.7  0.1   0:35   0 cat
7193 maxwell   25   0   400  400   352 R    22.7  0.1   0:35   0 cat
7191 maxwell   25   0   400  400   352 R    21.9  0.1   0:36   0 cat
7194 maxwell   25   0   404  404   352 R    21.9  0.1   0:35   0 cat
1184 einstein  15   0 11140 8708  3144 R     4.7  3.4   1:28   0 gnome-termina
7198 maxwell   39  19   404  404   352 R N❶  2.1❷ 0.1   0:02   0 simulation
4293 maxwell   15   0  5164 5016  2384 S     1.7  1.9   6:07   0 gnome-system-
```

- El comando **renice** puede utilizarse para cambiar el niceness de un proceso en ejecución.

```
[maxwell@station maxwell]$ renice 19 7347
7348: old priority 0, new priority 19
```

© 2007 Linux. Todos los derechos reservados.

Hacer procesos mucho más ambiciosos

- Los usuarios normales no pueden *bajar* el niceness de un proceso. Esto trae dos implicaciones:
  - Debido a que los procesos inician con un niceness de 0 por defecto, los usuarios normales no pueden hacer procesos "ambiciosos" con valores de niceness negativos.
  - Una vez que a un proceso le han bajado el nice, los usuarios normales no pueden volverlo "normal" otra vez .
- El administrador puede utilizar el comando renice como root para elevar el niceness
  - Usuario normal no puede restaurar dicha situación.

© 2007 Linux. Todos los derechos reservados.

Resumen de nice y renice

- Cada proceso tiene fijada una prioridad que Linux usa para determinar que intervalos de tiempo son compartidos. Puede fijar la prioridad de un proceso comenzando con el comando nice:

  $ nice -n 10 ./simulation &

- El comando nice puede cambiar la prioridad de un proceso cuando lo inicia. Si usted quiere cambiar la prioridad de amabilidad de un proceso corriendo, use renice :

  $ renice 10 641

  641: old priority 0, new priority 10

© 2007 Linux. Todos los derechos reservados.

Envio de señales

- **Conceptos Claves**
  - Las señales son una forma de bajo nivel de la comunicación entre procesos que surgen de una variedad de recursos, incluyendo el kernel, la terminal y otros procesos.
  - Las señales se distinguen por los números de señales que tienen nombres y usos simbólicos.
  - Los nombres simbólicos para los nombres de señales pueden listarse con el comando **kill -l**.
  - El comando **kill** envía señales a otros procesos.
  - Tras recibir una señal, un proceso puede ya sea, ignorarla, reaccionar de un modo especificado por defecto de kernel o implementar un manejador de señal personalizado.
  - Convencionalmente, el número de señal 15 (SIGTERM) se utiliza para solicitar la terminación de un proceso.
  - La señal número 9 (SIGKILL) termina un proceso y no puede anularse.
  - Los comandos **pkill** y **killall** pueden utilizarse para enviar señales a procesos especificados por nombre de comando o el usuario a quienes pertenecen.
  - Otras utilidades, tales como **top** y el Monitor de sistema GNOME, también pueden utilizarse para enviar señales.

© 2007 Linux. Todos los derechos reservados.

Señales

- Linux utiliza señales para notificar procesos de eventos anormales, y como un mecanismo comunicación entre procesos.
  - Como si el proceso tratase de dividir un número por 0, o
  - Tratase de accesar memoia que no le pertenece.
- Los procesos también pueden enviar señales a otros procesos.
- Los diferentes tipos de señales tienen números simbólicos pero también se identifican con números enteros.
- Los varios números enteros y el nombre simbólico que les son asignados pueden listarse por medio del comando **kill -l**

© 2007 Linux. Todos los derechos reservados.

¿Porque se envian las señales?

Hay una variedad de razones por las cuales las señales se pueden enviar a un proceso como se ilustra con los siguientes ejemplos.

- Excepciones de Hardware
  - El proceso le pidió al hardware realizar alguna operación errónea. Por ejemplo, el kernel enviará un proceso SIGFPE (señal número 8) si realiza una división por 0.
- Condiciones de Software
  - Los procesos pueden necesitar ser notificados de alguna condición anormal del software. Por ejemplo, cada vez que muere un proceso, el kernel envía un SIGCHLD (número de señal 17) al padre del proceso.
- Interrupciones de Terminal
  - Varias secuencias de control de teclas de la terminal envían señales al proceso de la shell **bash**. Por ejemplo, **CONTROL-C** envía un SIGINT (número de señal 2) mientras que **CONTROL-Z** envía un SIGTSTP (número de señal 20).
- Otros Procesos
  - Los procesos pueden elegir enviar señales a cualquier otro proceso perteneciente al mismo usuario. El comando **kill** está diseñado para hacer justo esto.

© 2007 Linux. Todos los derechos reservados.

Enviando Señales

- El comando **kill** se utiliza para enviar señales a otros procesos.

```
[einstein@station einstein]$ ps
  PID TTY          TIME CMD
 7985 pts/5    00:00:00 bash
 8060 pts/5    00:00:01 xclock
 8061 pts/5    00:00:00 ps
[einstein@station einstein]$ kill -17 8060
[einstein@station einstein]$ kill -CHLD 8060
[einstein@station einstein]$ kill -SIGCHLD 8060
```

© 2007 Linux. Todos los derechos reservados.

Recepción de Señales

- Cuando un proceso recibe una señal puede realizar una de las siguientes tres acciones.
  - Implementar un manejador de señal predeterminado del kernel
    - Cada señal se asigna una de las siguientes conductas.
      - » **Terminar**: El proceso de recepción se cierra.
      - » **Ignorar**: El proceso de recepción ignora la señal
      - » **Nucleo**: El proceso de recepción termina, pero bota una imagen de su memoria en un archivo llamado core en el actual directorio de trabajo del proceso. El archivo core puede ser utilizado por desarrolladores para ayudar a depurar el programa.
      - » **Parar**: Detener (suspender) el proceso.
  - Escoja ignorar la señal
    - Los programadores pueden elegir que su aplicación simplemente ignore señales especificadas.
  - Escoger implementar un manejador de señal personalizado
    - Los programadores pueden elegir implementar su propia conducta cuando se recibe una señal específica.

© 2007 Linux. Todos los derechos reservados.

Uso de Señales para terminar procesos

Señales importantes para usuarios normales

Número	Symbol	Acción
2	SIGINT	Interrumpe (solicita terminar) el proceso. Esta es la señal generada por la secuencia de control **CTRL-C**.
9	SIGKILL	Fuerza el terminar el proceso (esta señal puede *no* ser anulada por el proceso)
15	SIGTERM	Solicita la terminación del proceso.
20	SIGTSTP	Detiene (suspende) el proceso. Esta es una señal generada por la secuencia de control **CTRL-Z**.

© 2007 Linux. Todos los derechos reservados.

El comando pkill

- El comando **pkill** se puede utilizar para enviar señales a procesos seleccionados por medios más generales.

Opciones para el comando pkill

Opción	Efecto
-n	Selecciona únicamente los procesos coincidentes iniciados más recientemente.
-u *USER*	Selecciona procesos pertenecientes al usuario *USER*.
-t *TERM*	Selecciona procesos controlados por terminal *TERM*.

© 2007 Linux. Todos los derechos reservados.

El comando killall

- **killall** envía señales a procesos especificados por el nombre de comando.

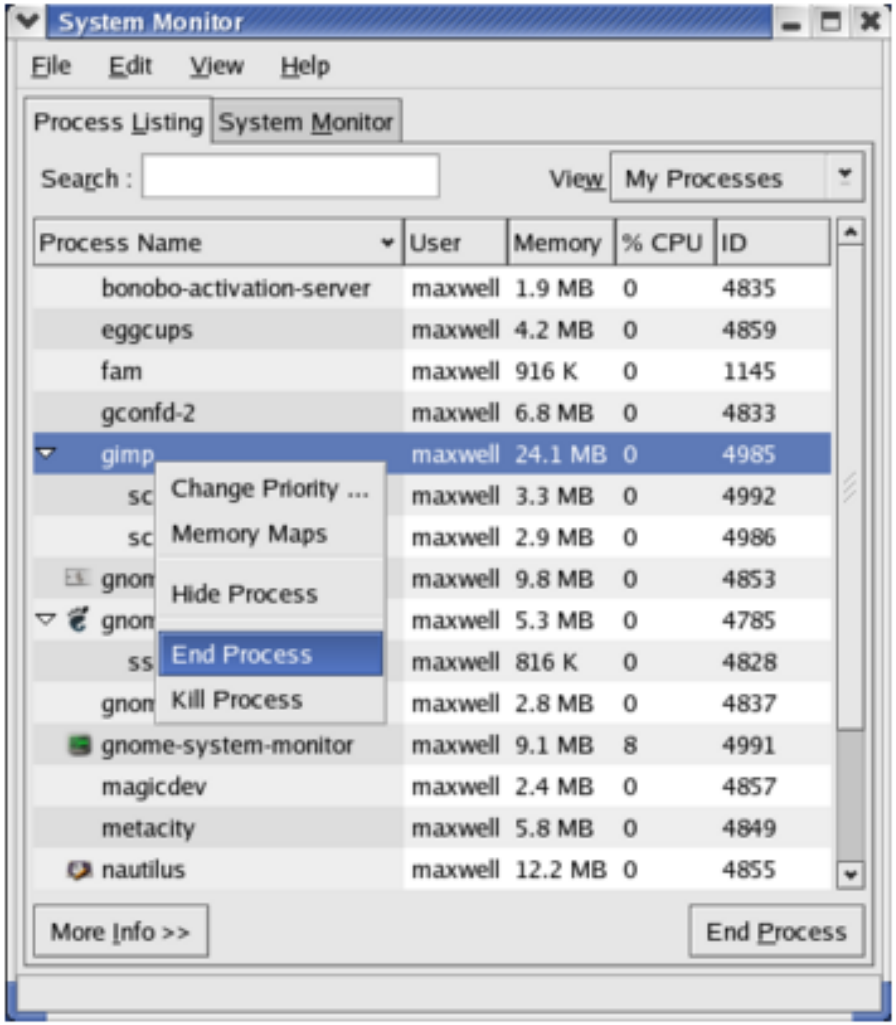
Opciones para el comando killall

Opción	Efecto
-i, --interactive	De modo interactivo pregunta al usuario antes de enviar la señal a un proceso.
-w, --wait	Espera hasta que todos los procesos estén muertos antes de retornar.

© 2007 Linux. Todos los derechos reservados.

El Monitor de Sistema

- La aplicación del Monitor de sistema GNOME, presentada en una lección anterior, también puede utilizarse para enviar señales a los procesos.
- Al hacer click derecho en un proceso, un menú emergente le permite al usuario seleccionar End Process, el cual tiene el efecto de entregar un SIGTERM al proceso.



© 2007 Linux. Todos los derechos reservados.

Control de Trabajo

- **Conceptos Claves**
  - La shell **bash** permite a los comandos ejecutarse en segundo plano como "trabajos".
  - La shell **bash** permite a un trabajo ejecutarse en segundo plano y puede tener múltiples trabajos en segundo plano.
  - El comando **jobs** listará todos los trabajos en segundo plano.
  - La secuencia de teclas CONTROL-Z suspenderá y enviará a segundo plano el actual trabajo que se encuentra en primer plano.
  - El comando **bg** reanuda un trabajo de segundo plano.
  - El comando **fg** trae un trabajo de segundo plano a primer plano.

© 2007 Linux. Todos los derechos reservados.

Ejecución de comandos en segundo plano como trabajos

- Cualquier comando que usted especifique puede también ejecutarse en el segundo plano,adjuntándole el signo (“&”) a la línea de comandos.
- Sólo los comandos de larga ejecución que no requieren entradas desde el teclado y no generan grandes cantidades de salida son apropiados para un segundo plano.

```
[einstein@station einstein]$ find / -size +1024k > bigfiles.txt 2> /dev/null &
[1] 7022
```

© 2007 Linux. Todos los derechos reservados.

Administración de múltiples trabajos

- Iniciando multiples jobs en background

```
[einstein@station einstein]$ ls
bin  sim_a  sim_b  sim_c  sim_d
[einstein@station einstein]$ ./sim_a &
[1] 7309
[einstein@station einstein]$ ./sim_b &
[2] 7311
[einstein@station einstein]$ ./sim_c &
[3] 7313
[einstein@station einstein]$ ./sim_d &
[4] 7315
```

- Visualizando jobs en background (comando jobs)

```
[einstein@station einstein]$ jobs
[1]    Running                 ./sim_a &
[2]    Running                 ./sim_b &
[3]-   Running                 ./sim_c &
[4]+   Running                 ./sim_d &
```

- Traer un trabajo al primer plano con fg (comando fg)

```
[einstein@station einstein]$ fg 3
./sim_c
```

© 2007 Linux. Todos los derechos reservados.

Suspensión del trabajo de primer plano con CONTROLZ

- La secuencia **CONTROL-Z** es un método para suspender procesos.

```
[einstein@station einstein]$ fg 3
./sim_c
CTRL-Z


[3]+  Stopped                 ./sim_c
[einstein@station einstein]$ jobs
[1]   Running                 ./sim_a &
[2]   Running                 ./sim_b &
[3]+  Stopped                 ./sim_c
[4]-  Running                 ./sim_d &
```

© 2007 Linux. Todos los derechos reservados.

Reiniciar un trabajo detenido en el segundo plano

- Un trabajo detenido puede reiniciarse en el segundo plano con el comando incorporado **bg**.

```
[einstein@station einstein]$ bg 3
[3]+ ./sim_c &
[einstein@station einstein]$ jobs
[1]    Running                 ./sim_a &
[2]    Running                 ./sim_b &
[3]-   Running                 ./sim_c &
[4]+   Running                 ./sim_d &
```

© 2007 Linux. Todos los derechos reservados.

Matar Trabajos (Killing Jobs)

- El comando **kill**, utilizado para entregar señales para procesos se implementa como un comando incorporado de shell.

```
[einstein@station einstein]$ jobs
[1]    Running                 ./sim_a &
[2]    Running                 ./sim_b &
[3]-   Running                 ./sim_c &
[4]+   Running                 ./sim_d &
[einstein@station einstein]$ kill 2  ❶
-bash: kill: (2) - Operation not permitted
[einstein@station einstein]$ kill %2 ❷
[einstein@station einstein]$
[2]    Terminated              ./sim_b
[einstein@station einstein]$ jobs
[1]    Running                 ./sim_a &
[3]-   Running                 ./sim_c &
[4]+   Running                 ./sim_d &
```

© 2007 Linux. Todos los derechos reservados.

Administración de trabajos en la shell bash

Comando	Acción
trabajos	Lista todos los trabajos
fg [*N*]	Trae el trabajo *N* de segundo plano al primer plano (por defecto, el trabajo de segundo plano "actual").
CTRL-Z	Suspende y envía al segundo plano el actual comando de primer plano
bg [*N*]	Inicia el trabajo de segundo plano detenido *N* (por defecto, el trabajo "actual"en segundo plano).
kill %*N*	Termina el trabajo de segundo plano *N* (enviando la señal SIGTERM).

© 2007 Linux. Todos los derechos reservados.

Programación de Taeas retrasadas: at

- **Conceptos Claves**
  - El comando **at** puede someter comandos para que se ejecuten más tarde.
  - El comando **batch** puede emitir comandos para que se ejecuten cuando la carga de las máquinas sea baja.
  - Los comandos pueden escribirse directamente o someterse como un script.
  - la stdout de los trabajos at se envía por correo al usuario.
  - Los comandos **atq** y **atrm** se utilizan para examinar y quitar trabajos actualmente programados.

© 2007 Linux. Todos los derechos reservados.

El demonio atd

- El demonio **atd** le permite a los usuarios someter trabajos para ser realizados más tarde
- Para utilizar el demonio **atd**, éste debe estar ejecutándose.
- Los usuarios pueden confirmar que **atd** se está ejecutando simplemente al examinar la lista de procesos en ejecución

```
[madonna@station madonna]$ ps aux | grep atd
daemon    4730  0.0  0.2  1420  532 ?        S    15:42   0:00 /usr/sbin/atd
madonna   5570  0.0  0.2  3572  640 pts/2    S    16:43   0:00 grep atd
```

© 2007 Linux. Todos los derechos reservados.

Envío de trabajos con at

- El comando **at** se utiliza para someter tra
- bajos al demonio **atd** para que se ejecuten en una hora específica. Los comandos que se van a ejecutar son sometidos ya sea como script (con la opción **-f**) o escritos directamente via la stdin.
- **at** [-f filename | -m] TIME

Opción	Efecto
-f filename	ejecuta el script especificado por el nombre de archivo
-m	Notifica al usuario por correo electrónico cuando se ejecuta, incluso si no hay salida.

```
[hogan@station hogan]$ at 2:00 am
warning: commands will be executed using (in order) a) $SHELL b) login shell c)
/bin/sh
at> lpr fanmail.txt
at> CTRL-D
job 7 at 2003-06-17 02:00
```

```
[hogan@station hogan]$ atq
7       2003-06-17 02:00 a hogan
```

```
[hogan@station hogan]$ atrm 7
[hogan@station hogan]$ atq
[hogan@station hogan]$ lpr fanmail.txt
```

© 2007 Linux. Todos los derechos reservados.

Resumen de los comandos at

command	uso
atd	El demonio que ejecuta trabajos sometidos. Los usuarios no utilizan el comando **atd** directamente.
at	Somete trabajos al demonio **atd** para que se ejecuten en un tiempo específico.
batch	Somete trabajos al demonio **atd** para que se ejecuten cuando el sistema no esté ocupado.
atq	Lista trabajos en espera con el demonio **atd**.
atrm	Cancela un trabajo en espera con el demonio **atd** antes de ejecutarse.

© 2007 Linux. Todos los derechos reservados.

Programación de Taeas periodicas: cron

- **Conceptos Claves**
  - La utilidad cron se utiliza para programar tareas recurrentes.
  - El comando **crontab** provee un frontend para editar archivos crontab.
  - El archivo crontab utiliza 5 campos para especificar la información de temporización.
  - la stdout de trabajos cron se envía por correo al usuario.

© 2007 Linux. Todos los derechos reservados.

The cron Service

- El demonio **crond** es el demonio que realiza tareas periódicamente en nombre del sistema o de usuarios individuales.
- El demonio suele iniciarse cuando el sistema arranca, por lo tanto, la mayoría de usuarios lo pueden ignorar. Al listar todos los procesos y buscar *crond* puede confirmar si el demonio **crond** está en ejecución.

```
[hogan@station hogan]$ ps aux | grep crond
root       891  0.0  0.0  1572  128 ?        S    Jun17   0:00 crond
hogan     8118  0.0  0.2  3576  644 pts/2    S    10:15   0:00 grep crond
```

© 2007 Linux. Todos los derechos reservados.

Usando el comando crontab

- Los usuarios rara vez administran su archivo crontab directamente (o incluso saben dónde se almacena), en cambio, utilizan el comando crontab para editar la lista o quitarla.
  - **crontab** {-e | -l | -r}
  - **crontab** FILE

Opción	Efecto
-e	modifica el archivo actual
-l	lista el archivo actual
-r	quita el archivo actual

```
[hogan@station hogan]$ crontab -l
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (/tmp/crontab.9212 installed on Wed Jun 18 11:46:36 2003)
# (Cron version -- $Id: 040_crontab.dbk,v 1.1.1.1 2003/11/26 18:53:02 bowe Exp $)
# set the default language to be english.
LOCALE=en_US

05 *  *  *  *  procinfo
15 04 *  *  *  find $HOME -size +100k
25 04 1  *  *  echo "Pay your bills"
35 04 14 3  *  echo "Beware the Ides of March" | mail -s "a warning" julius
45 04 *  *  1  find $HOME -atime +30 | lpr
[hogan@station hogan]$ crontab -l > mycopy
```

© 2007 Linux. Todos los derechos reservados.

Sintaxis crontab

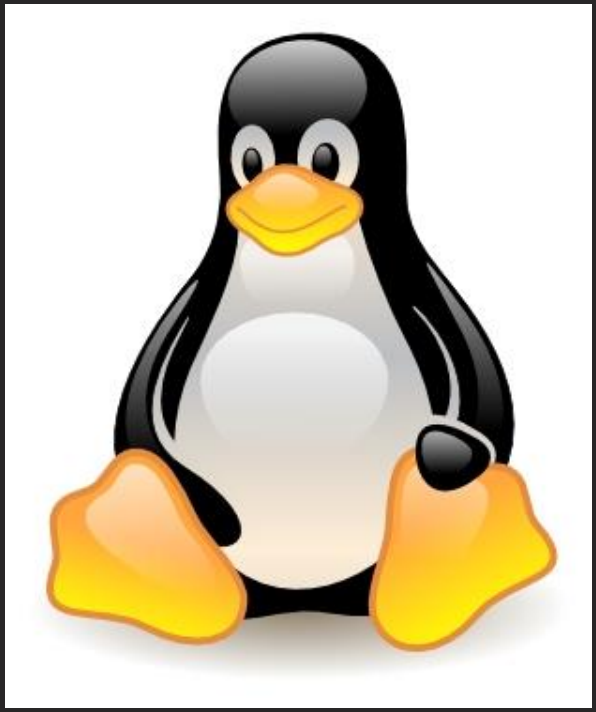
- Los usuarios especifican los trabajos que se van a ejecutar y cuándo se van a ejecutar, al configurar un archivo conocido como el "cuadro cron" a menudo abreviado en inglés "crontab".
  - Las líneas del comando crontab constan de seis campos separados de espacios en blanco. Los primeros 5 campos se utilizan para especificar cuándo ejecutar el comando y el sexto campo se utiliza para especificar el comando a ejecutar (compuesto de todo después del quinto campo).

```
# set the default language to be english
LOCALE=en_US

05 *  *  *  *  procinfo
15 04 *  *  *  find $HOME -size +100k
25 04 1  *  *  echo "Pay your bills"
35 04 14 3  *  echo "Beware the Ides of March" | mail -s "a warning" julius
45 04 *  *  1  find $HOME -atime +30 | lpr
```

```
      ,----------------->  minute
      |  ,-------------->  hour
      |  |  ,----------->  day of month
      |  |  |  ,------->  month (1=January, 2=February, ...)
      |  |  |  |  ,---->  day of week (0=Sunday, 1=Monday, ...)
      |  |  |  |  |  ,->  command to run
      |  |  |  |  |  |
      25 04 1  *  *  echo "Pay your bills"
```

© 2007 Linux. Todos los derechos reservados.



redhat Linux